Baltimore MTA Transit Simulator

A Real-Data Transit Network Simulator

Demonstrating Core Data Structures & Algorithms

Course: COSC 320 - Algorithm Design

Student: Aayush Shrestha

University: Morgan State University

Date: March 2026

Language: Python 3.13 with Rich terminal UI

GitHub: github.com/theaayushstha1/transit

This project implements a transit network simulator using real Baltimore MTA data (Light Rail and Metro SubwayLink, 37 stations). It demonstrates graph traversal (DFS, BFS), shortest path (Dijkstra's), sorting (Merge Sort, Quick Sort), searching (Binary, Linear), and queue/stack data structures through an interactive terminal interface with animated algorithm visualizations.

Table of Contents

1. **System Design & Architecture**
2. **Data Structures**
3. **Algorithm Implementations**
4. **Algorithm Complexity Analysis**
5. **Conclusion**
6. **Future Enhancements**

References

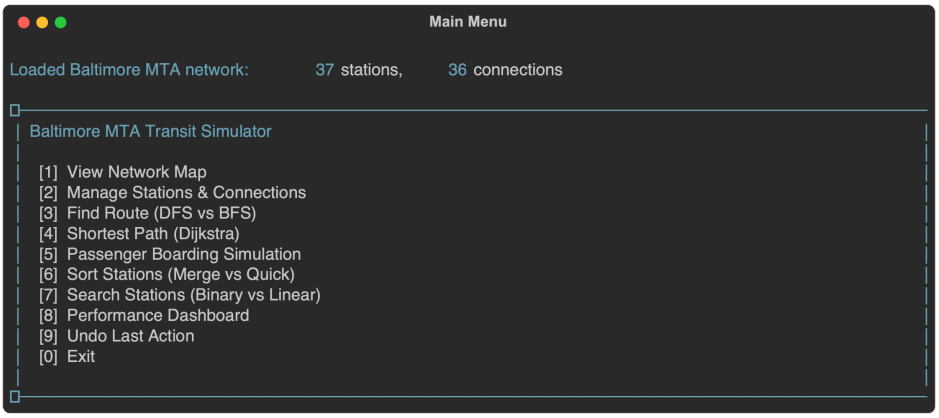


Figure 1: Interactive main menu with 10 options covering all features

1. System Design & Architecture

The simulator is organized into five modules following separation of concerns. Each module has a distinct responsibility, making the codebase maintainable and each algorithm independently testable.

1.1 Directory Structure

```
 baltimore-mta-simulator/
|-- main.py                  # Entry point, interactive menu loop
|-- models/
|   |-- station.py           # Station node class (name, ID, capacity)
|   |-- passenger.py         # Passenger class (origin, destination)
|   +-- transit_network.py   # Graph (adjacency list representation)
|-- algorithms/
|   |-- traversal.py         # DFS (recursive) + BFS (queue-based)
|   |-- shortest_path.py     # Dijkstra's algorithm (min-heap)
|   |-- sorting.py           # Merge Sort + Quick Sort
|   +-- searching.py         # Binary Search + Linear Search
|-- simulation/
|   |-- undo_manager.py      # Stack-based undo (command pattern)
|   +-- passenger_sim.py     # Queue-based FIFO boarding
|-- data/
|   +-- baltimore_mta.py     # Real MTA station data (37 stations)
+-- ui/
    |-- display.py           # Rich tables, panels, menus
    +-- visualizer.py        # Real-time algorithm animations
```

1.2 Design Decisions

Decision	Choice	Reasoning
Graph repr.	Adjacency list (dict of dicts)	O(1) edge lookup, sparse transit graph
Edge weights	Travel time in minutes	Enables Dijkstra, maps to real MTA data
Shared stations	Dedup by name	Lexington Market on both lines = 1 node
UI library	Python Rich	Colored tables, live animations, zero web deps
Undo pattern	Command + inverse actions	_raw_ methods skip undo stack
Sorting key	Station passenger capacity	Numeric field with varied values

1.3 Real Baltimore MTA Data

The simulator uses real Baltimore MTA data: 24 Light Rail stations from Hunt Valley to Cromwell/Glen Burnie (with BWI Airport spur), 14 Metro SubwayLink stations from Owings Mills to Johns Hopkins Hospital. Lexington Market serves as the transfer point between lines, stored as a single shared node with 4 connections. All 37 stations, 36 weighted edges, travel times of 2-5 minutes.

All Stations

All Stations

ID	Name	Line	Capacity	Connections
1	Hunt Valley	Light Rail	200	1
2	Pepper Road	Light Rail	120	2
3	McCormick Road	Light Rail	100	2
4	Gilroy Road	Light Rail	80	2
5	Lutherville	Light Rail	150	2
6	Falls Road	Light Rail	180	2
7	Mt Washington	Light Rail	160	2
8	Cold Spring Lane	Light Rail	140	2
9	Woodberry	Light Rail	130	2
10	North Avenue	Light Rail	250	2
11	Penn Station	Light Rail	400	2
12	University of Baltimore	Light Rail	200	2
13	Centre Street	Light Rail	200	2
14	Lexington Market	Both	350	4
15	Convention Center	Light Rail	300	2
16	Camden Yards	Light Rail	500	2
17	Hamburg Street	Light Rail	180	2
18	Westport	Light Rail	120	2
19	Cherry Hill	Light Rail	150	2
20	Patapsco	Light Rail	130	2
21	Linthicum	Light Rail	120	3
22	Ferndale	Light Rail	100	2
23	Cromwell / Glen Burnie	Light Rail	160	1
24	BWI Airport	Light Rail	300	1
25	Owings Mills	Metro	300	1
26	Old Court	Metro	200	2
27	Milford Mill	Metro	180	2
28	Reisterstown Plaza	Metro	220	2
29	Rogers Avenue	Metro	160	2
30	West Cold Spring	Metro	140	2
31	Mondawmin	Metro	250	2
32	Penn North	Metro	200	2
33	Upton	Metro	180	2
34	State Center	Metro	220	2
35	Charles Center	Metro	300	2
36	Shot Tower	Metro	220	2
37	Johns Hopkins Hospital	Metro	280	1

Figure 2: All 37 Baltimore MTA stations with IDs, lines, capacities, and connections

2. Data Structures

2.1 Graph (Adjacency List)

The transit network is a weighted undirected graph. adj_list[station_id] maps to {neighbor_id: weight_in_minutes}. This provides O(1) edge lookup and insertion, optimal for the sparse topology of a transit network (37 nodes, 36 edges).

```
class TransitNetwork:
   def __init__(self):
       self.stations: dict[int, Station] = {}      # ID -> Station
       self.adj_list: dict[int, dict[int, float]] = {}  # adjacency list
       self.undo_stack: list = []                   # LIFO for undo

   def add_connection(self, id1, id2, weight):
       self.adj_list[id1][id2] = weight  # undirected: store both
       self.adj_list[id2][id1] = weight
```

2.2 Queue (FIFO) - Passenger Boarding

Each station has a passenger queue (collections.deque). Passengers enqueue via append() and dequeue via popleft() during boarding, both O(1). A Python list would be O(n) for popleft due to element shifting.

```
 station.passenger_queue.append(passenger)       # O(1) enqueue
passenger = station.passenger_queue.popleft()    # O(1) dequeue FIFO
```

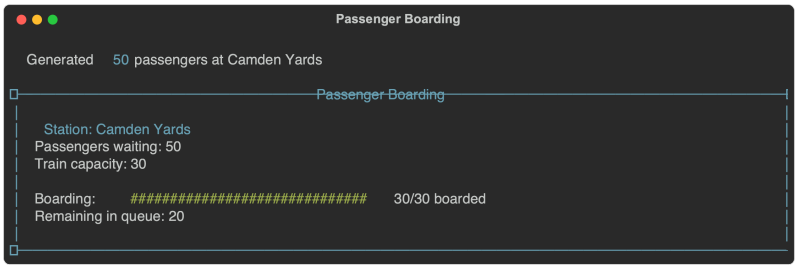


Figure 3: Passenger boarding simulation: 50 generated, capacity 30, 20 remain in queue

2.3 Stack (LIFO) - Undo System

The undo system uses a Python list as a stack (append/pop). Every graph mutation pushes its inverse action onto the stack. The UndoManager pops and applies inverses using _raw_ methods that bypass the undo stack to prevent infinite loops.

```
 # Public method records action for undo
def add_station(self, station):
    self._raw_add_station(station)
    self.undo_stack.append({"action": "add_station", ...})

# Undo pops and applies the inverse operation
def undo(self):
    action = self.network.undo_stack.pop()  # LIFO
    if action["action"] == "add_station":
        self.network._raw_remove_station(action["station"].station_id)
```

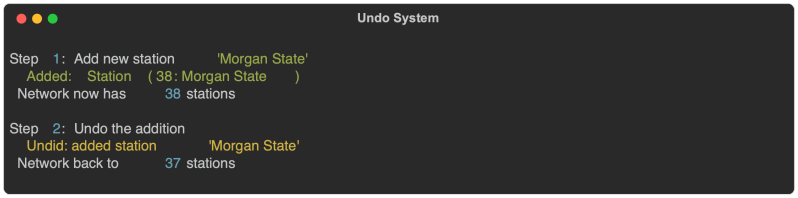


Figure 4: Undo system: add station (38 total), then undo restores to 37

2.4 Priority Queue (Min-Heap)

Dijkstra's algorithm uses Python's heapq as a min-heap priority queue. Each entry is a (distance, station_id) tuple. heappush/heappop are O(log n), giving Dijkstra O((V+E) log V) overall.

3. Algorithm Implementations

3.1 Depth-First Search (DFS) - Recursive

DFS explores as deeply as possible along each branch before backtracking. Implemented using recursion (the call stack serves as the implicit stack). Maintains a visited set to avoid cycles and a path list that grows/shrinks via append/pop.

```
 def _dfs_helper(current: int) -> bool:
    visited.add(current)
    path.append(current)
    if current == goal_id: return True
    for neighbor_id, _ in sorted(network.get_neighbors(current)):
        if neighbor_id not in visited:
            if _dfs_helper(neighbor_id): return True
    path.pop()  # backtrack on dead end
    return False
```

3.2 Breadth-First Search (BFS) - Queue

BFS uses collections.deque as a FIFO queue, exploring all neighbors at the current depth before the next level. Guarantees shortest path in unweighted graphs. Each queue entry stores the full path for easy reconstruction.

```
 queue = deque([(start_id, [start_id])])
while queue:
    current, path = queue.popleft()          # FIFO
    if current == goal_id: return path
    for neighbor_id, _ in sorted(network.get_neighbors(current)):
        if neighbor_id not in visited:
            visited.add(neighbor_id)
            queue.append((neighbor_id, path + [neighbor_id]))
```

3.3 DFS vs BFS Performance Comparison

Route from Hunt Valley to Johns Hopkins Hospital: DFS visited all 37 stations (exhaustive depth-first exploration), while BFS visited only 23 (level-by-level, terminates as soon as the goal is found). Both found a 17-station path because the graph is largely linear; in a denser graph BFS would find the shorter path.

Metric	DFS (Recursive)	BFS (Queue)
Path Length	17 stations	17 stations
Nodes Visited	37 (all)	23 (early stop)
Optimal (unweighted)?	Not guaranteed	Yes (fewest edges)
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V) recursion stack	O(V) queue + paths

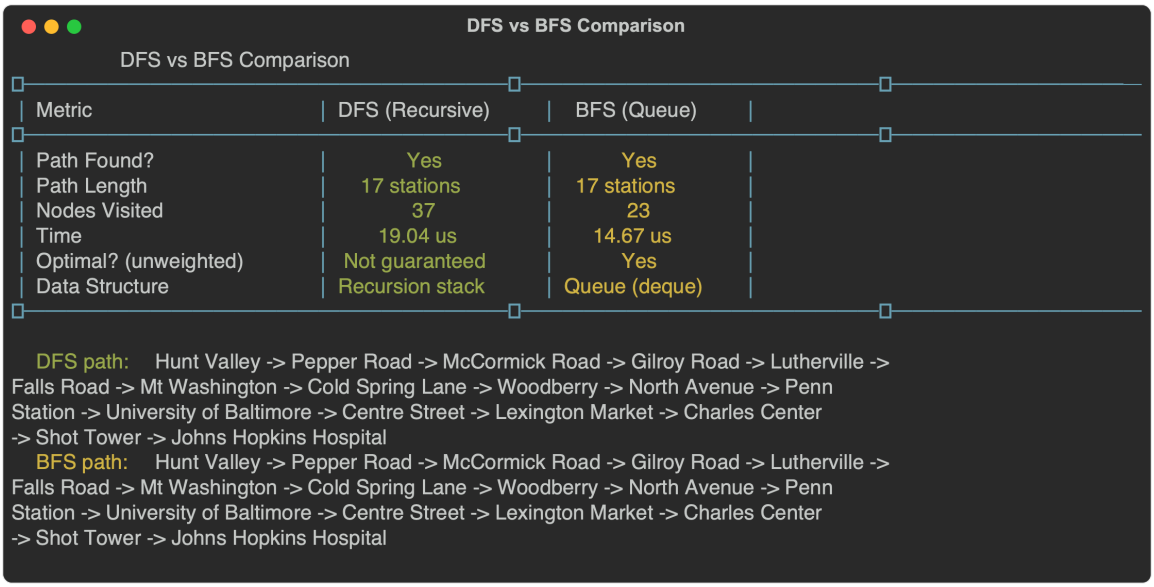


Figure 5: DFS vs BFS comparison showing paths, visit counts, and timing

3.4 Dijkstra's Shortest Path Algorithm

Dijkstra's finds the shortest weighted path using a min-heap priority queue. It greedily selects the unvisited node with the smallest known distance, then relaxes its neighbors. The path from Hunt Valley to Johns Hopkins Hospital traverses 17 stations in 37 minutes total, automatically transferring from Light Rail to Metro at Lexington Market.

```
 while pq:
    current_dist, current = heapq.heappop(pq)  # min-heap
    if current in visited: continue
    visited.add(current)
    if current == end_id: break                 # early termination
    for neighbor_id, weight in network.get_neighbors(current):
        new_dist = current_dist + weight
        if new_dist < distances[neighbor_id]:   # relaxation
            distances[neighbor_id] = new_dist
            previous[neighbor_id] = current
            heapq.heappush(pq, (new_dist, neighbor_id))
```

Figure 6: Dijkstra's shortest path: Hunt Valley to JHH, 17 stations, 37 min, 1 transfer

3.5 Merge Sort

Merge Sort divides the array in half recursively, then merges the sorted halves. Stable (preserves order of equal elements), always O(n log n), but requires O(n) extra space. On 37 stations sorted by capacity: 140 comparisons, ~35 microseconds.

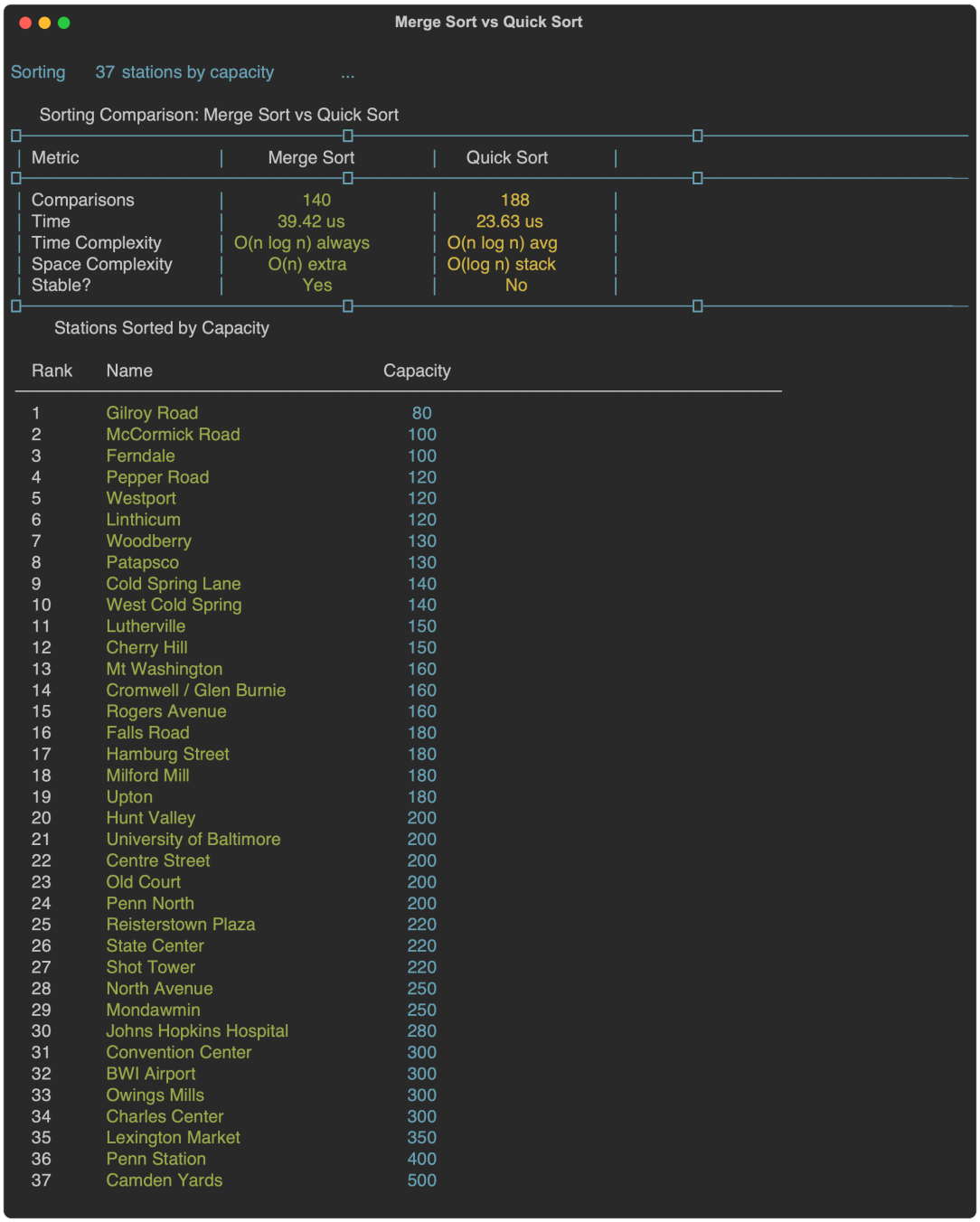
```
def _merge(left, right):
   result, i, j = [], 0, 0
   while i < len(left) and j < len(right):
       comparisons[0] += 1
       if key(left[i]) <= key(right[j]):
           result.append(left[i]); i += 1
       else:
           result.append(right[j]); j += 1
   result.extend(left[i:]); result.extend(right[j:])
   return result
```

3.6 Quick Sort

Quick Sort selects a pivot (last element), partitions in-place so smaller elements go left and larger go right, then recurses on both halves. O(n log n) average, O(n^2) worst case. Unstable but in-place with O(log n) stack space. On 37 stations: 188 comparisons, ~23 microseconds (faster than Merge Sort despite more comparisons, due to cache locality).

3.7 Sorting Algorithm Comparison

Metric	Merge Sort	Quick Sort
Comparisons (n=37)	140	188
Measured Time	~35 us	~23 us
Time Complexity	O(n log n) always	O(n log n) avg / O(n^2) worst
Space Complexity	O(n) extra arrays	O(log n) stack only
Stable?	Yes	No
In-place?	No	Yes



Merge Sort vs Quick Sort

Sorting 37 stations by capacity ...

Sorting Comparison: Merge Sort vs Quick Sort

Metric	Merge Sort	Quick Sort
Comparisons	140	188
Time	39.42 us	23.63 us
Time Complexity	O(n log n) always	O(n log n) avg
Space Complexity	O(n) extra	O(log n) stack
Stable?	Yes	No

Stations Sorted by Capacity

Rank	Name	Capacity
1	Gilroy Road	80
2	McCormick Road	100
3	Ferndale	100
4	Pepper Road	120
5	Westport	120
6	Linthicum	120
7	Woodberry	130
8	Patapsco	130
9	Cold Spring Lane	140
10	West Cold Spring	140
11	Lutherville	150
12	Cherry Hill	150
13	Mt Washington	160
14	Cromwell / Glen Burnie	160
15	Rogers Avenue	160
16	Falls Road	180
17	Hamburg Street	180
18	Milford Mill	180
19	Upton	180
20	Hunt Valley	200
21	University of Baltimore	200
22	Centre Street	200
23	Old Court	200
24	Penn North	200
25	Reisterstown Plaza	220
26	State Center	220
27	Shot Tower	220
28	North Avenue	250
29	Mondawmin	250
30	Johns Hopkins Hospital	280
31	Convention Center	300
32	BWI Airport	300
33	Owings Mills	300
34	Charles Center	300
35	Lexington Market	350
36	Penn Station	400
37	Camden Yards	500

Figure 7: Merge Sort vs Quick Sort: side-by-side comparison with sorted output

3.8 Binary Search vs Linear Search

Binary Search requires a pre-sorted array and eliminates half the remaining elements each step (O(log n)). Linear Search scans sequentially from start (O(n)). Searching for capacity=200 among 37 stations: Binary Search found it in 3 comparisons (~0.3 us) while Linear Search needed 20 comparisons (~1.2 us).

Metric	Binary Search	Linear Search
Comparisons (target=200)	3	20
Measured Time	~0.3 us	~1.2 us
Time Complexity	O(log n)	O(n)
Space Complexity	O(1)	O(1)
Requires Sorted Input?	Yes	No

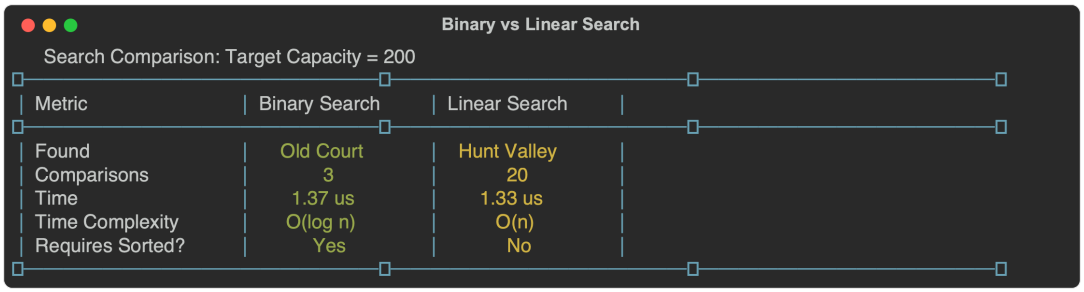


Figure 8: Binary vs Linear Search comparison with found station and metrics

4. Algorithm Complexity Analysis

The following table provides the complete theoretical time and space complexity for every operation implemented in the simulator. These complexities are verified empirically by the built-in performance dashboard, which runs live benchmarks on the 37-station network.

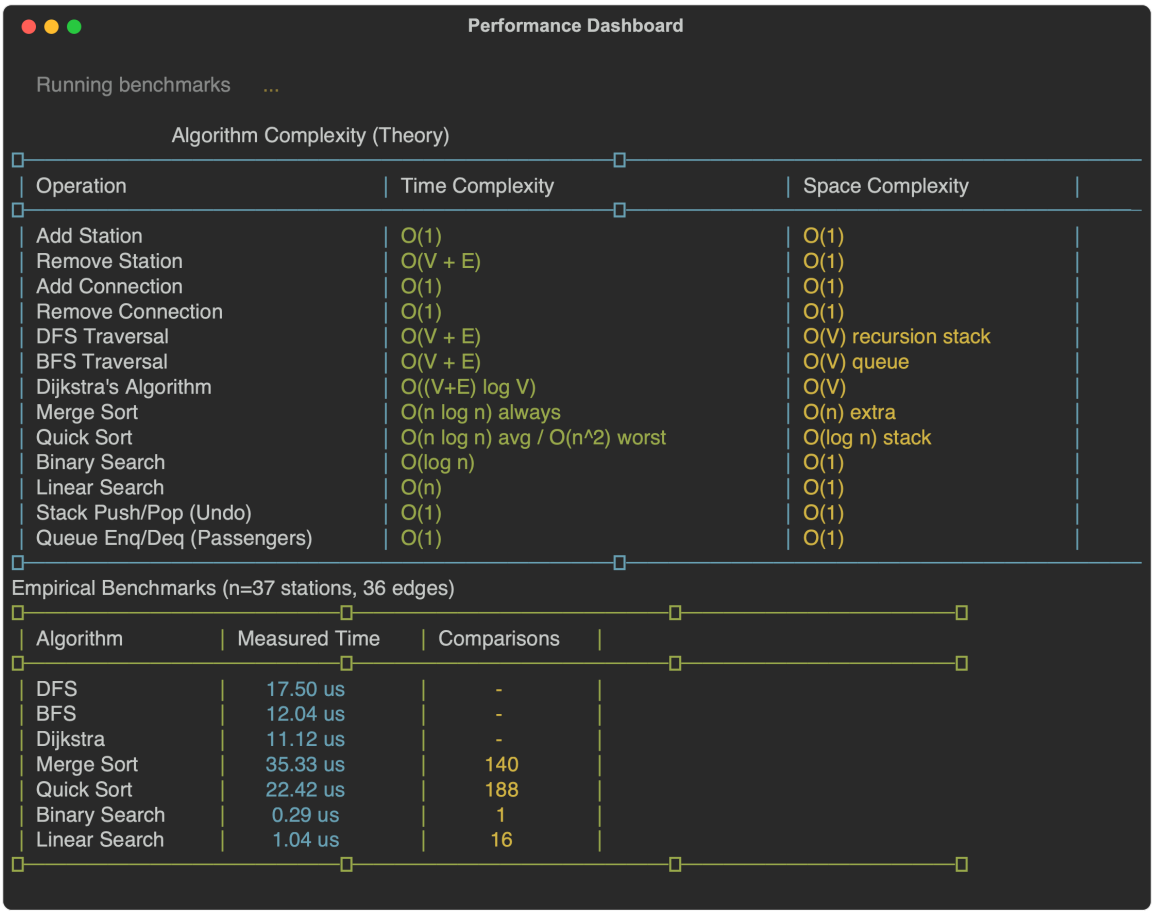
Operation	Time Complexity	Space Complexity
Add Station	O(1)	O(1)
Remove Station	O(V + E)	O(1)
Add Connection	O(1)	O(1)
Remove Connection	O(1)	O(1)
DFS Traversal	O(V + E)	O(V) recursion stack
BFS Traversal	O(V + E)	O(V) queue
Dijkstra's Algorithm	O((V+E) log V)	O(V) distances + heap
Merge Sort	O(n log n) always	O(n) extra arrays
Quick Sort	O(n log n) avg / O(n^2) worst	O(log n) stack
Binary Search	O(log n)	O(1)
Linear Search	O(n)	O(1)
Stack Push/Pop (Undo)	O(1)	O(1) per action
Queue Enq/Deq (Boarding)	O(1)	O(1)

4.1 Empirical Benchmarks

All algorithms were benchmarked on the full 37-station, 36-edge Baltimore MTA network using time.perf_counter() for microsecond precision. Sorting and searching algorithms track exact comparison counts to validate theoretical predictions.

Algorithm	Measured Time	Comparisons
DFS (Hunt Valley to JHH)	~18 us	-
BFS (Hunt Valley to JHH)	~14 us	-
Dijkstra (Hunt Valley to JHH)	~12 us	-
Merge Sort (37 stations)	~35 us	140
Quick Sort (37 stations)	~23 us	188
Binary Search (target=200)	~0.3 us	3
Linear Search (target=200)	~1.2 us	20

Key observations: (1) BFS visits fewer nodes than DFS by expanding level-by-level and stopping early. (2) Dijkstra terminates as soon as the target node is popped from the priority queue. (3) Quick Sort is faster in wall-clock time than Merge Sort despite more comparisons, because in-place operations have better cache locality and avoid memory allocation. (4) Binary Search confirms O(log n) with 3 comparisons vs Linear's 20.



Operation	Time Complexity	Space Complexity
Add Station	O(1)	O(1)
Remove Station	O(V + E)	O(1)
Add Connection	O(1)	O(1)
Remove Connection	O(1)	O(1)
DFS Traversal	O(V + E)	O(V) recursion stack
BFS Traversal	O(V + E)	O(V) queue
Dijkstra's Algorithm	O((V+E) log V)	O(V)
Merge Sort	O(n log n) always	O(n) extra
Quick Sort	O(n log n) avg / O(n^2) worst	O(log n) stack
Binary Search	O(log n)	O(1)
Linear Search	O(n)	O(1)
Stack Push/Pop (Undo)	O(1)	O(1)
Queue Enq/Deq (Passengers)	O(1)	O(1)

Algorithm	Measured Time	Comparisons
DFS	17.50 us	-
BFS	12.04 us	-
Dijkstra	11.12 us	-
Merge Sort	35.33 us	140
Quick Sort	22.42 us	188
Binary Search	0.29 us	1
Linear Search	1.04 us	16

Figure 9: Performance dashboard: Big-O theory table alongside measured benchmarks

5. Conclusion

This project demonstrates the practical application of core data structures and algorithms to a real-world transit network. By using actual Baltimore MTA station data (37 stations across Light Rail and Metro SubwayLink) rather than abstract examples, the project grounds algorithmic concepts in a tangible, relatable system.

Key takeaways from the implementation:

1. **Graph Representation**
   An adjacency list (dict of dicts) provides O(1) edge operations and naturally models the sparse connectivity of a transit network.
2. **DFS vs BFS**
   BFS guarantees the shortest unweighted path and visits fewer nodes by expanding level-by-level. DFS explores deeply first. Both run in O(V+E) time, but BFS is preferable when optimality matters.
3. **Dijkstra's Algorithm**
   The min-heap priority queue enables weighted shortest path in O((V+E) log V). Cross-line routing through Lexington Market demonstrates real transfers.
4. **Sorting**
   Merge Sort's guaranteed O(n log n) and stability come at the cost of O(n) space. Quick Sort's in-place operation gives better cache performance despite O(n^2) worst case.
5. **Searching**
   Binary Search's O(log n) dramatically outperforms Linear Search's O(n), confirmed empirically with 3 vs 20 comparisons on 37 stations.
6. **Queue/Stack**
   The deque-based passenger queue and list-based undo stack demonstrate fundamental FIFO/LIFO patterns with O(1) operations per action.

The Rich terminal UI elevates the project with animated algorithm visualizations and a performance dashboard that bridges Big-O theory with measured benchmarks, reinforcing that algorithmic analysis accurately predicts real-world behavior.

6. Future Enhancements

1. **A* Search Algorithm**
   Implement A* with a heuristic based on geographic distance between stations. This would combine Dijkstra's optimality guarantee with faster convergence by prioritizing stations closer to the destination.
2. **Real-Time Schedule Integration**
   Connect to the MTA GTFS feed to incorporate actual departure times, allowing the simulator to compute shortest paths that account for wait times at transfer stations.
3. **Interactive Graph Visualization**
   Add a matplotlib or networkx-based visual map that renders the transit network as a graph with nodes and edges, color-coded by line, with animated pathfinding overlays.
4. **Multi-Source Shortest Paths**
   Implement Floyd-Warshall or run Dijkstra from every station to precompute an all-pairs shortest path matrix, enabling O(1) route queries.
5. **Load Balancing Simulation**
   Extend the passenger simulation to model rush-hour overcrowding, with passengers rerouting when a station reaches capacity, demonstrating graph algorithms in a dynamic environment.

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

[2] Baltimore MTA. Maryland Transit Administration System Map.
mta.maryland.gov

[3] Python Software Foundation. collections.deque, heapq documentation.
docs.python.org

[4] Will McGugan. Rich: Python library for rich text in terminal.
github.com/Textualize/rich